lgcns-fe-next · 2026-10-29 · Next.js 16.4.3 · React 19.3.0 · Tailwind CSS 4

오늘 진행한 Next.js 작업과 연결된 공식 문서

저장소에 있는 소스만 기준으로 골랐습니다. 데이터 페칭, 서버 액션, 미들웨어처럼 오늘 코드에 없는 Next.js 기능은 넣지 않았습니다.

> shadcn/ui는 `package.json`에 없습니다. 3부는 설치 문서와, 오늘 만든 빈 버튼·네비게이션이 나중에 바꿀 공식 컴포넌트를 연결한 별도 정리입니다.

소스와 기술의 연결

오늘 한 일	소스	기술	공식 문서
주소는 `/`, `/login` 인데 폴더 이름으로 구역을 나눔	`app/(main)/`, `app/(auth)/`	App Router, Route Groups	Project structure, Route Groups
홈은 헤더를 쓰고, 로그인은 다른 레이아웃	`app/layout.tsx`, `app/(main)/layout.tsx`, `app/(auth)/layout.tsx`	중첩 layout, `children`	Layouts and Pages, layout
메뉴와 로고 클릭으로 이동	`DesktopNavigation.tsx`, `MobileNavigation.tsx`, `LogoItem.tsx`	`next/link`	Linking and Navigating, Link
모바일 메뉴만 클릭으로 열고 닫음	`MobileNavigation.tsx`	`"use client"`, `useState`	Server and Client Components, use client
외부 고양이 이미지, LG CNS 로고	`BannerSwiperItem.tsx`, `LogoItem.tsx`, `next.config.ts`	`next/image`, `images.remotePatterns`	Image, images 설정
Google 폰트를 CSS 변수로 넘김	`app/layout.tsx`, `app/globals.css`	`next/font/google`, Metadata	Font, Metadata
`@/component/...` import	`tsconfig.json`, `app/(main)/layout.tsx`	경로 별칭 `@/*`	TypeScript
커밋 전 ESLint	`.husky/pre-commit`, `eslint.config.mjs`	eslint-config-next, Husky	ESLint, Husky
설정에 캐시·프리페치·Turbopack CSS 로더	`next.config.ts`	`cacheComponents`, `partialPrefetching`, `turbopack`	각 next.config 문서
제목 색을 props가 덮어씀	`CustomTitle.tsx`, `BannerSwiperItem.tsx`	Tailwind 유틸리티, tailwind-merge	Tailwind theme / utilities, tailwind-merge
데스크톱 메뉴는 md 이상, 햄버거는 그 미만	`DesktopNavigation.tsx`, `MobileNavigation.tsx`	반응형 variant `md:`	Responsive design
버튼·네비는 아직 직접 작성. shadcn 미설치	`Button.tsx`, `DesktopNavigation.tsx`, `MobileNavigation.tsx`	shadcn/ui Button, Navigation Menu, Sheet	shadcn installation, Tailwind v4, 해당 컴포넌트

컴포넌트는 `app` 밖에 두었습니다. `component/atoms`, `molecules`, `organisms`, `templates` 는 라우트가 아닙니다. Next.js는 라우트 파일이 아닌 UI를 `app` 밖이나 비공개 폴더에 두라고 안내합니다.

1. Next.js 공식 문서

버전은 프로젝트에 설치된 16.4.3 문서 기준입니다. 아래 주소는 nextjs.org의 App Router 문서입니다.

프로젝트 구조

문서 Project structure and organization

https://nextjs.org/docs/app/getting-started/project-structure

소스 `app/`, `component/`

폴더가 주소를 만들고, 라우트가 아닌 파일은 URL에 안 올라갑니다. 오늘 화면 컴포넌트는 `app` 밖 `component/`에 있으므로 `/component`라는 주소는 생기지 않습니다. `(main)`, `(auth)`처럼 괄호 폴더는 같은 문서의 Route groups 절과 아래 Route Groups 문서가 설명합니다.

페이지와 레이아웃

문서 Layouts and Pages

https://nextjs.org/docs/app/getting-started/layouts-and-pages

이어서 page · layout

https://nextjs.org/docs/app/api-reference/file-conventions/page

https://nextjs.org/docs/app/api-reference/file-conventions/layout

소스 `app/(main)/page.tsx` → `/`, `app/(auth)/login/page.tsx` → `/login`, `app/layout.tsx`, `app/(main)/layout.tsx`, `app/(auth)/layout.tsx`

`page.tsx`의 default export가 그 주소의 화면입니다. `layout.tsx`는 `children`으로 페이지를 감쌉니다. 이동해도 레이아웃은 유지됩니다.

실제 감싸기 순서는 이렇습니다. 루트 `app/layout.tsx`가 `html`과 `body`를 만듭니다. `/`는 그 안에 `(main)/layout.tsx`의 `Header`가 한 번 더 감쌉니다. `/login`은 `Header` 대신 `(auth)/layout.tsx`를 탑니다. 루트 레이아웃의 `LayoutProps<"/">`는 이 layout 문서와 TypeScript 문서가 설명하는, 라우트에 맞춰 생성되는 props 타입입니다.

라우트 그룹

문서 Route Groups

https://nextjs.org/docs/app/api-reference/file-conventions/route-groups

소스 `app/(main)/`, `app/(auth)/`

괄호 폴더 이름은 URL에 들어가지 않습니다. `app/(auth)/login/page.tsx`의 주소는 `/auth/login`이 아니라 `/login`입니다. 그룹을 나눈 이유는 같은 깊이에서 레이아웃을 다르게 쓰기 위해서입니다. 메인에는 헤더, 인증 화면에는 `AuthLayout`만 붙습니다.

링크와 이동

문서 Linking and Navigating · Link

https://nextjs.org/docs/app/getting-started/linking-and-navigating

https://nextjs.org/docs/app/api-reference/components/link

소스 `component/molecules/DesktopNavigation.tsx`, `component/molecules/MobileNavigation.tsx`, `component/molecules/LogoItem.tsx`

앱 안 이동은 `<a>` 대신 `Link`를 씁니다. 데스크톱 메뉴는 `/`, `/about`, `/contact`로 연결됩니다. 오늘 `app` 안에 있는 페이지는 `/`와 `/login`뿐입니다. `/about`과 `/contact`용 `page.tsx`는 아직 없습니다.

로고의 `Link`는 `href="https://www.lgcns.com/"`와 `target="_blank"`입니다. 외부 주소도 `Link`로 열 수 있고, 이건 앱 내부 클라이언트 이동이 아닙니다.

서버 컴포넌트와 클라이언트 컴포넌트

문서 Server and Client Components · use client

https://nextjs.org/docs/app/getting-started/server-and-client-components

https://nextjs.org/docs/app/api-reference/directives/use-client

소스 `component/molecules/MobileNavigation.tsx`만 `"use client"`

App Router 컴포넌트는 기본이 서버 컴포넌트입니다. `useState`처럼 브라우저 상태가 필요하면 파일 맨 위에 `"use client"`를 붙입니다. 모바일 메뉴의 `isOpen`이 그 경우입니다. `Header`, `DesktopNavigation`, `BannerSwiperItem`에는 이 지시어가 없어서 서버 컴포넌트로 남습니다. 클라이언트 파일이 서버 컴포넌트를 import하면 그 경계가 따라오므로, 상태가 필요한 파일만 클라이언트 경계를 두는 것이 이 문서의 요지입니다.

이미지

문서 Image · images 설정 · 호스트 미설정 오류

https://nextjs.org/docs/app/getting-started/images

https://nextjs.org/docs/app/api-reference/components/image

https://nextjs.org/docs/app/api-reference/config/next-config-js/images

https://nextjs.org/docs/messages/next-image-unconfigured-host

소스 `component/molecules/BannerSwiperItem.tsx`, `component/molecules/LogoItem.tsx`, `next.config.ts`의 `images.remotePatterns`

`next/image`의 `src`가 외부 주소이면, 그 호스트를 `remotePatterns`에 적어야 합니다. 오늘 `i.pinimg.com`이 빠져 있어서 런타임 오류가 났고, `protocol`, `hostname`, `pathname: "/**"`를 넣어 해결했습니다. 같은 배열에 `www.lgcns.com`도 있습니다. 로고 `Image`가 이 호스트의 SVG를 가리킵니다.

공식 Image 문서는 SVG를 기본으로 최적화하지 않습니다. SVG를 허용하려면 `images.dangerouslyAllowSVG`가 따로 필요합니다. 로고 소스는 SVG이고, 이 설정은 `next.config.ts`에 아직 없습니다.

`width`와 `height`는 배너(1000)와 로고(100)에 모두 넘깁니다. 정적 import가 아닌 문자열 `src`에는 이 두 값이 필요합니다. `component/atoms/LogoImage.tsx`는 `next/image`가 아니라 `<img>`라서 이 제한을 타지 않습니다. 지금 로고 화면은 `LogoItem`이 그리고, `LogoImage`는 아직 연결되지 않았습니다.

폰트와 메타데이터

문서 Font · Metadata and OG images · generateMetadata

https://nextjs.org/docs/app/getting-started/fonts

https://nextjs.org/docs/app/api-reference/components/font

https://nextjs.org/docs/app/getting-started/metadata-and-og-images

https://nextjs.org/docs/app/api-reference/functions/generate-metadata

소스 `app/layout.tsx`, `app/globals.css`

`next/font/google`의 `Geist`, `Geist_Mono`, `Gothic_A1`, `Black_Han_Sans`가 CSS 변수 이름을 만듭니다. `html`의 `className`에는 Black Han Sans, Geist, Geist Mono 변수가 붙어 있습니다. Gothic A1은 변수는 만들지만 `className`에는 아직 없습니다. `globals.css`의 `h1`은 `var(--font-black-han-sans)`를 쓰고, `@theme inline`은 Geist 변수를 Tailwind의 `--font-sans`, `--font-mono`로 연결합니다. 폰트 문서와 Tailwind `@theme` 문서가 이 한 줄에서 만납니다.

같은 파일의 `export const metadata`가 문서 제목과 설명을 바꿉니다. 지금 값은 create-next-app 기본값인 "Create Next App"입니다. 페이지마다 바꾸려면 해당 `page.tsx`나 `layout.tsx`에서 `metadata` 또는 `generateMetadata`를 둡니다.

CSS를 앱에 넣는 방법

문서 CSS

https://nextjs.org/docs/app/getting-started/css

소스 `app/layout.tsx`의 `import "./globals.css"`, `app/globals.css`

전역 CSS는 루트 레이아웃에서 한 번 import합니다. 이 프로젝트는 그 파일 첫 줄 `@import "tailwindcss"`로 Tailwind v4를 올립니다. Next.js CSS 문서의 Tailwind 절이 이 설치 형태를 설명합니다. v3용 별도 가이드(`/docs/app/guides/tailwind-v3-css`)는 오늘 설정과 버전이 다릅니다.

TypeScript 경로 별칭

문서 TypeScript

https://nextjs.org/docs/app/api-reference/config/typescript

소스 `tsconfig.json`의 `"@/*": ["./*"]`, `app/(main)/layout.tsx`의 `import Header from "@/component/templates/Header"`, `app/(main)/page.tsx`

`@/`는 프로젝트 루트입니다. `app` 안의 페이지가 `component`를 상대 경로 `../../` 없이 가져옵니다. `component` 폴더 안에서는 아직 상대 경로(`../organisms/Logo`)를 씁니다. 둘 다 같은 파일을 가리키며, 별칭은 `tsconfig`의 `paths`와 Next TypeScript 설정이 맞춰 줍니다.

ESLint와 커밋 훅

Next 문서 ESLint

https://nextjs.org/docs/app/api-reference/config/eslint

Husky 문서 Get started

https://typicode.github.io/husky/get-started.html

소스 `eslint.config.mjs`, `package.json`의 `lint`·`prepare`, `.husky/pre-commit`

`eslint.config.mjs`는 `eslint-config-next`의 core-web-vitals와 typescript 설정을 펼칩니다. Next ESLint 문서가 이 구성을 설명합니다. Husky는 Next 공식 도구가 아닙니다. Git 루트가 `lgcns-fe-next`의 상위 폴더라서 `prepare`가 `cd .. && husky lgcns-fe-next/.husky`로 훅 경로를 맞춥니다. `pre-commit`은 저장소 루트에서 `lgcns-fe-next`로 들어가 `npm run lint`를 실행합니다.

next.config.ts에 켜 둔 항목

문서 cacheComponents · partialPrefetching · turbopack · agentFeedback

https://nextjs.org/docs/app/api-reference/config/next-config-js/cacheComponents

https://nextjs.org/docs/app/api-reference/config/next-config-js/partialPrefetching

https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack

https://nextjs.org/docs/app/api-reference/config/next-config-js/agentFeedback

소스 `next.config.ts`

- `cacheComponents: true`는 `use cache`와 정적 셸을 켭니다. 오늘 컴포넌트에는 `use cache`가 아직 없습니다. 설정만 켜진 상태입니다.
- `partialPrefetching: true`는 `cacheComponents`와 같이 둡니다. 공식 문서는 이 값을 비우면 경고가 난다고 합니다. `Link`의 프리페치 동작이 이 설정의 영향을 받습니다.
- `turbopack.rules`는 `*.css`를 `@tailwindcss/turbopack` 로더로 보냅니다. 패키지에도 `@tailwindcss/turbopack`가 있습니다. Turbopack 문서는 CSS를 기본 지원하므로, 이 로더 규칙은 그 기본 위에 프로젝트에서 추가한 설정입니다.
- `experimental.agentFeedback`는 오늘 화면 기능과 무관합니다. 설정 파일에 켜져 있어 문서 주소만 적습니다.

2. Tailwind CSS 공식 문서

설치된 버전은 Tailwind CSS 4입니다. `app/globals.css` 첫 줄이 `@import "tailwindcss"` 입니다. v3의 `@tailwind base/components/utilities` 세 줄 방식은 이 프로젝트에 없습니다.

Next.js에 설치하기

문서 Install Tailwind CSS with Next.js · Tailwind CSS v4

https://tailwindcss.com/docs/installation/framework-guides/nextjs

https://tailwindcss.com/blog/tailwindcss-v4

Next에서 같은 설명 https://nextjs.org/docs/app/getting-started/css

소스 `package.json`의 `tailwindcss`, `@tailwindcss/turbopack`, `app/globals.css`, `app/layout.tsx`

v4는 CSS 한 줄 import로 시작합니다. 템플릿 경로를 `tailwind.config`에 적지 않아도 클래스를 찾습니다. 이 저장소에는 `tailwind.config.js`가 없습니다.

테마 변수와 @theme inline

문서 Theme variables

https://tailwindcss.com/docs/theme

소스 `app/globals.css`의 `@theme inline`, `app/layout.tsx`의 `next/font`

`@theme`에 적은 CSS 변수가 유틸리티 클래스를 만듭니다. 예를 들어 `--color-background`는 색 유틸리티와 연결됩니다. `inline`은 테마 값을 별도 변수로 한 번 더 복사하지 않고, 이미 있는 변수를 그대로 가리키게 합니다. 공식 테마 문서의 예가 `@theme inline { --font-sans: var(--font-inter); }` 형태입니다.

오늘 코드는 그 예와 같습니다. `next/font`가 `--font-geist-sans`를 만들고, `@theme inline`이 `--font-sans: var(--font-geist-sans)`로 받습니다. `--color-background`와 `--color-foreground`도 `:root` 값을 가리킵니다. 다크 모드는 `prefers-color-scheme`로 `:root` 값만 바꿉니다.

반응형

문서 Responsive design · Display · Flex · Gap

https://tailwindcss.com/docs/responsive-design

https://tailwindcss.com/docs/display

https://tailwindcss.com/docs/flex

https://tailwindcss.com/docs/gap

소스 `DesktopNavigation.tsx`의 `hidden md:flex`, `MobileNavigation.tsx`의 `md:hidden`

접두사 없는 클래스는 모든 너비에 적용되고, `md:`는 768px(`48rem`) 이상에서만 적용됩니다. 모바일 전용 스타일을 만들 때 `sm:`을 "작은 화면"으로 쓰면 안 됩니다. 작은 화면은 접두사 없는 클래스이고, `md:`가 그 위를 덮습니다.

데스크톱 목록은 기본 `hidden`이었다가 `md:flex`로 나타납니다. 햄버거와 펼친 메뉴는 `md:hidden`이라 768px 이상에서는 숨습니다. `items-center`와 `gap-4`는 flex 컨테이너의 정렬과 간격입니다.

오늘 클래스와 유틸리티 문서

클래스	소스	문서
`text-2xl`, `font-bold`, `text-center`, `text-white`, `text-red-500`, `text-black`	`CustomTitle.tsx`, `BannerSwiperItem.tsx`, `(auth)/layout.tsx`	https://tailwindcss.com/docs/font-size https://tailwindcss.com/docs/font-weight https://tailwindcss.com/docs/text-align https://tailwindcss.com/docs/color
`bg-white`	`app/(auth)/layout.tsx`	https://tailwindcss.com/docs/background-color
`min-h-full`, `flex`, `flex-col`, `h-full`, `w-full`, `w-6`, `h-6`	`app/layout.tsx`, `LogoImage.tsx`, `MobileNavigation.tsx`	https://tailwindcss.com/docs/min-height https://tailwindcss.com/docs/flex-direction https://tailwindcss.com/docs/width https://tailwindcss.com/docs/height
`object-cover`	`LogoImage.tsx`	https://tailwindcss.com/docs/object-fit
`antialiased`	`app/layout.tsx`	https://tailwindcss.com/docs/font-smoothing
`sr-only`	`MobileNavigation.tsx` 버튼 문구 "Open main menu"	https://tailwindcss.com/docs/screen-readers

`sr-only` 는 화면에서는 숨기고 스크린 리더에는 읽히게 합니다. 아이콘 버튼에 이름만 남기는 패턴이고, 접근성 문서와도 맞습니다.

클래스 우선순위와 tailwind-merge

문서 tailwind-merge (Tailwind 공식 패키지는 아님)

https://github.com/dcastil/tailwind-merge

소스 `component/atoms/CustomTitle.tsx`, `component/atoms/LogoImage.tsx`, `package.json` 의 `tailwind-merge`

Tailwind 유틸리티는 클래스 문자열에 나중에 적었다고 이기지 않습니다. 같은 속성이면 생성된 CSS 안에서의 순서가 이깁니다. 그래서 `text-white ${className}` 처럼 이어 붙이면, 밖에서 `text-red-500` 을 넘겨도 흰색이 남을 수 있습니다.

`twMerge("text-2xl font-bold text-center text-white", className)` 는 겹치는 유틸리티만 나중 값으로 바꿉니다. `BannerSwiperItem` 이 `className="text-red-500"` 를 넘기면 결과 클래스는 `text-2xl font-bold text-center text-red-500` 입니다. 크기, 굵기, 정렬은 기본값이 남습니다. `LogoImage` 도 `w-full h-full object-cover` 위에 같은 방식으로 `className` 을 올립니다.

3부의 shadcn `cn()` 은 이 `twMerge` 에 `clsx` 를 더한 함수입니다. 지금 코드는 `clsx` 없이 `twMerge` 만 씁니다.

3. shadcn/ui 공식 문서

> 이 프로젝트에는 shadcn/ui가 설치되어 있지 않습니다. `components.json`, `@radix-ui`, `class-variance-authority`, `lucide-react` 가 없습니다. 아래는 오늘 만든 파일 중 나중에 shadcn 컴포넌트로 바꿀 자리와, 그 전에 읽을 설치 문서입니다.

Next.js에 설치

문서 Installation · Next.js · Manual · components.json · Theming

https://ui.shadcn.com/docs/installation

https://ui.shadcn.com/docs/installation/next

https://ui.shadcn.com/docs/installation/manual

https://ui.shadcn.com/docs/components-json

https://ui.shadcn.com/docs/theming

이미 맞춰진 소스 `tsconfig.json`의 `@/*`, `app/globals.css`의 `@import "tailwindcss"`, 루트 레이아웃의 CSS import

공식 Next.js 설치는 App Router, Tailwind, `@/*` 별칭을 전제로 합니다. 이 세 가지는 오늘 프로젝트에 있습니다. `npx shadcn@latest init`을 실행하면 `components.json`과 보통 `lib/utils.ts`의 `cn`이 생깁니다. `cn`은 `clsx`로 클래스를 모으고 `tailwind-merge`로 충돌을 정리합니다. `CustomTitle`이 이미 `twMerge`를 직접 쓰고 있으므로, shadcn을 넣을 때 그 호출을 `cn`으로 모을 수 있습니다.

Tailwind v4와의 관계

문서 Tailwind v4

https://ui.shadcn.com/docs/tailwind-v4

소스 Tailwind 4, React 19.3, `@theme inline`이 이미 `globals.css`에 있음

shadcn은 Tailwind v4의 `@theme`과 `@theme inline`을 지원한다고 이 문서에 적혀 있습니다. 새 프로젝트의 기본 스타일은 `new-york`이고, 색은 OKLCH를 씁니다. 오늘 `globals.css`의 색은 `#ffffff`, `#171717` 같은 직접 값입니다. shadcn 테마를 넣으면 이 파일의 `:root`와 `@theme`에 컴포넌트용 색 변수가 추가됩니다. 기존 `--background`, `--foreground` 이름과 겹치는지 그때 맞춰야 합니다.

오늘 소스와 대응되는 컴포넌트

오늘 소스	지금 상태	대응 문서	연결되는 기술
`component/atoms/Button.tsx`	빈 파일. 배너의 "View More"는 이 컴포넌트가 아니라 `BannerSwiperItem.tsx` 의 `<button>`	https://ui.shadcn.com/docs/components/button	Radix Slot, CVA variant, Tailwind. `variant` 와 `size` 로 스타일을 나눕니다. 지금 `CustomTitle` 이 `className` 으로 덮어쓰는 방식과 비슷하게, Button도 `className` 을 `cn()` 으로 받습니다.
`component/molecules/DesktopNavigation.tsx`	`ul` 과 `next/link` . `hidden md:flex`	https://ui.shadcn.com/docs/components/navigation-menu	Radix Navigation Menu. 링크 자체는 계속 `next/link` 를 넣을 수 있습니다. 반응형으로 숨기는 클래스는 Tailwind `md:` 가 담당하고, shadcn이 그 레이아웃을 대신하지는 않습니다.
`component/molecules/MobileNavigation.tsx`	`"use client"` , `useState` , 햄버거 SVG, `sr-only`	https://ui.shadcn.com/docs/components/sheet	Radix Dialog 기반의 옆면 패널. 열림 상태를 직접 `useState` 로 들고 있는 지금 구조를, Sheet의 `open` 으로 옮기는 자리입니다. 클라이언트 컴포넌트여야 하는 점은 Next `use client` 문서 그대로입니다.
`component/atoms/CustomTitle.tsx`	`twMerge` 로 제목 단계와 클래스를 받음	https://ui.shadcn.com/docs/installation/manual 의 `cn` 절	shadcn에 같은 제목 컴포넌트는 없습니다. 연결점은 클래스 병합뿐입니다. Typography 예시는 https://ui.shadcn.com/docs/components 목록의 별도 타이포그래피 컴포넌트가 아니라 테마의 글자 스타일입니다.

`SubTitle.tsx` 도 비어 있습니다. shadcn 문서에 1:1로 대응하는 컴포넌트는 없고, 설치 후 타이포그래피 클래스나 Button·제목 조합으로 채울 자리입니다.

넣기 전에 읽을 순서

1. Next.js 설치 문서. 별칭과 `globals.css` 가 이미 있는지 확인하는 절차입니다.
2. Tailwind v4 문서. `@theme inline` 과 기존 색 변수가 어떻게 합쳐지는지 봅니다.
3. Manual 설치의 `cn` . `CustomTitle` , `LogoImage` 의 `twMerge` 와 같은 역할입니다.
4. Button, Navigation Menu, Sheet. 빈 `Button.tsx` 와 두 네비게이션 파일을 바꿀 때 엽니다.